NOMBRE DE LA INSTITUCIÓN

Universidad Peruana Cayetano Heredia - Pontificia Universidad Católica del Perú

FACULTAD O ESCUELA

Facultad de Ingeniería Biomédica

TÍTULO DEL TRABAJO

METODOLOGÍA VDI : IMPLEMENTACIÓN

INTEGRANTES

ACOSTA PORTAL, LUIS JULIO
AGURTO ROMERO, ROGGER LEONARDO
AQUINO PEDRAZA, DIMAS LEANDRO
BAQUERIZO DAVILA, NATALIA ANDREA
GOMEZ VARGAS, ISABEL ANGELES
GUTIERREZ PACURI, GRECIA VALENTINA

DOCENTES

JHOMER CONTRERAS
IVAN CALLE
CARMEN LLANOS
LISBETH RAYMUNDO

CURSO

Fundamentos de Biodiseño

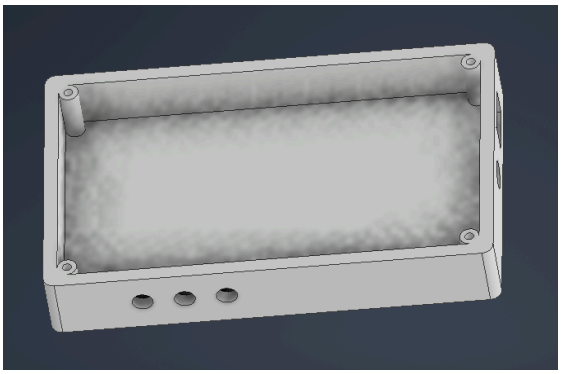
CICLO / SECCIÓN

4to ciclo

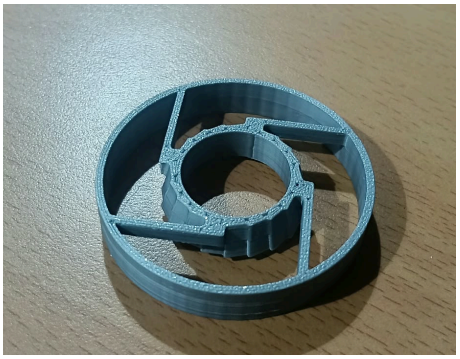
I.Implementación preliminar de los subsistemas según el diseño:

MECÁNICA Y FABRICACIÓN





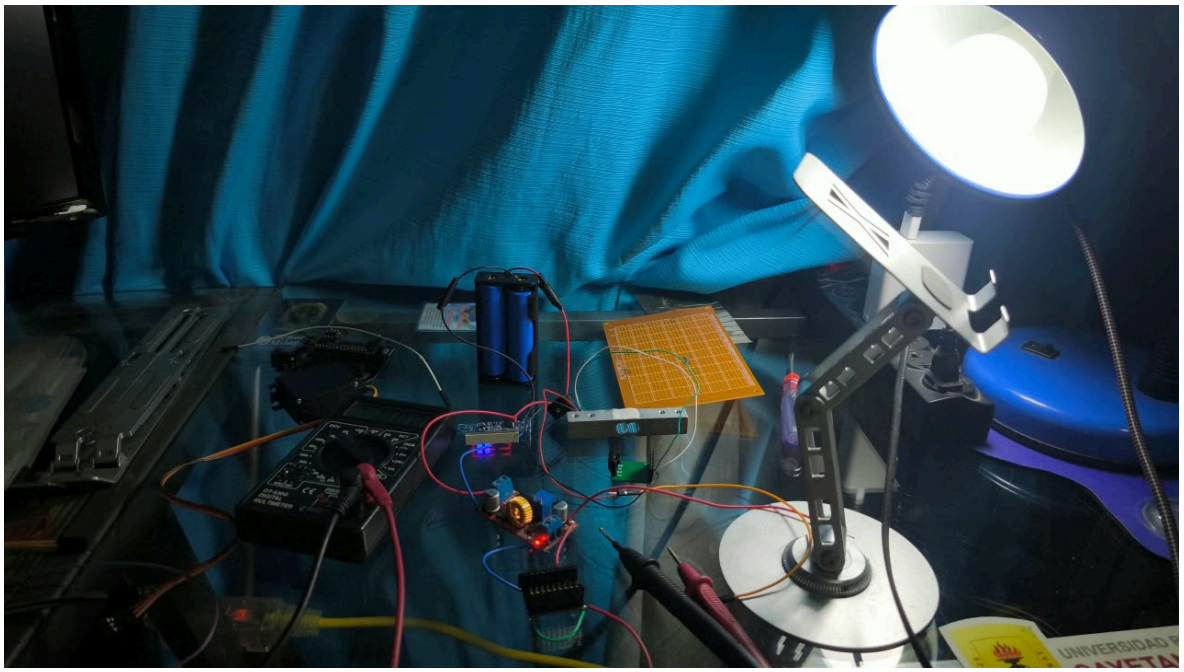
Caja de control: Se fabricó mediante impresión 3D para alojar y proteger los componentes electrónicos del sistema.





Trinquete: Se fabricó mediante impresión 3D como parte del mecanismo de bloqueo, permitiendo restringir el movimiento en una dirección.

ELECTRÓNICA Y SENSORES



Montaje electrónico

PROGRAMACIÓN Y CONTROL

```
void setup() {
  Serial.begin(115200);
  Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);

  bascula.begin(PIN_HX711_DOUT, PIN_HX711_SCK);
  bascula.set_scale(HX711_UNIDADES_POR_CUENTA);
  if (bascula.is_ready()) {
    Serial.println("HX711 listo. Retire toda carga: se tarará en 2 segundos.");
    delay(2000);
    bascula.tare(20);
  } else {
    Serial.println("ERROR: HX711 no responde; revisar DOUT, SCK y alimentación.");
  }

  if (!imu.begin(0x68, &Wire)) {
    Serial.println("ERROR: MPU6050 no responde en 0x68 (pruebe 0x69 si AD0=HIGH).");
  } else {
    imu.setAccelerometerRange(MPU6050_RANGE_4_G);
    imu.setGyroRange(MPU6050_RANGE_500_DEG);
    imu.setFilterBandwidth(MPU6050_BAND_21_HZ);
    Serial.println("MPU6050 listo. Mantenga la rodilla inmóvil para fijar reposo.");
  }

  servoRodilla.setPeriodHertz(50);
  servoRodilla.attach(PIN_SERVO, 500, 2500);
  servoRodilla.write((int)SERVO_CENTRO_GRADOS);
  anguloServoActual = SERVO_CENTRO_GRADOS;

  Serial.println("ESP32 listo y activo. El interruptor físico controla la alimentación.");
  Serial.println("El nivel de batería se consulta visualmente en el indicador 2S.");
}
```

Inicialización del sistema: Al encender el dispositivo, se inicializan y calibran los componentes electrónicos, como el sensor MPU6050 y el módulo HX711, además de preparar el servomotor para su funcionamiento antes de ejecutar el

programa principal.

```
void loop() {
  const uint32_t ahora = millis();
  if (ahora - ultimaLecturaMs < INTERVALO_LECTURA_MS) return;

  float anguloRelativo = 0.0f;
  const bool imuOk = leerAngulo(anguloRelativo);
  const bool hxOk = bascula.is_ready();
  float fuerza = NAN;
  if (hxOk) fuerza = bascula.get_units(3);

  if (hxOk) {
    if (!hayCarga && fuerza >= UMBRAL_CARGA) hayCarga = true;
    if (hayCarga && fuerza <= UMBRAL_CARGA - HISTERESIS_CARGA) hayCarga = false;
  }

  // Reproduce la decisión del flujograma: apoyo detectado y ángulo menor que 0°.
  if (sistemaActivo && hayCarga && hxOk && imuOk &&
      anguloRelativo < ANGULO_DISPARO_GRADOS) {
    const float objetivo = constrain(ANGULO_OBJETIVO_GRADOS,
                                     ANGULO_MINIMO_OBJETIVO_GRADOS,
                                     ANGULO_MAXIMO_GRADOS);
    const float servoObjetivo = constrain(
        SERVO_CENTRO_GRADOS + objetivo * SERVO_GRADOS_POR_GRADO_RODILLA,
        SERVO_MINIMO_GRADOS, SERVO_MAXIMO_GRADOS);
    anguloServoActual = servoObjetivo;
    servoRodilla.write((int)anguloServoActual);
  } else if (!hayCarga && sistemaActivo && hxOk) {
    // Sin apoyo, vuelve al centro para dejar listo el siguiente ciclo.
    anguloServoActual = SERVO_CENTRO_GRADOS;
    servoRodilla.write((int)anguloServoActual);
  }

  if (ahora - ultimoEstadoMs >= INTERVALO_ESTADO_MS) {
    ultimoEstadoMs = ahora;
    Serial.printf("modo=%s | HX711=%s | carga=%.2f | apoyo=%s | IMU=%s | angulo_rel=%.1f deg | servo=%d deg\n",
                  sistemaActivo ? "ACTIVO" : "REPOSO",
                  hxOk ? "OK" : "ERROR", fuerza,
                  hayCarga ? "SI" : "NO", imuOk ? "OK" : "ERROR",
                  anguloRelativo, (int)anguloServoActual);
  }
}
```

Funcionamiento del programa principal: El sistema realiza continuamente lecturas de la fuerza de pisada y del ángulo de inclinación de la rodilla. Cuando detecta apoyo y un ángulo menor a 0°, activa el servomotor para ajustar su posición. Al dejar de detectar apoyo, el servomotor regresa a su posición central, preparándose para el siguiente paso.

```
#include <Wire.h>
#include <HX711.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>

// Pines
constexpr int PIN_HX711_DOUT = 32;
constexpr int PIN_HX711_SCK  = 33;
constexpr int PIN_SERVO      = 18;
constexpr int PIN_I2C_SDA    = 21;
constexpr int PIN_I2C_SCL    = 22;

constexpr float HX711_UNIDADES_POR_CUENTA = 1.0f;
constexpr float UMBRAL_CARGA              = 5.0f;
constexpr float HISTERESIS_CARGA          = 1.0f;

constexpr float ANGULO_DISPARO_GRADOS = 0.0f;
constexpr float ANGULO_OBJETIVO_GRADOS = 25.0f;
constexpr float ANGULO_MINIMO_OBJETIVO_GRADOS = 20.0f;
constexpr float ANGULO_MAXIMO_GRADOS = 30.0f;
constexpr float ANGULO_MINIMO_GRADOS = 0.0f;

constexpr float SERVO_CENTRO_GRADOS = 90.0f;
constexpr float SERVO_GRADOS_POR_GRADO_RODILLA = 1.0f;
constexpr int SERVO_MINIMO_GRADOS = 10;
constexpr int SERVO_MAXIMO_GRADOS = 170;

constexpr uint32_t INTERVALO_LECTURA_MS = 20;
constexpr uint32_t INTERVALO_ESTADO_MS = 500;
constexpr float CONSTANTE_FILTRO_COMPLEMENTARIO = 0.98f;

HX711 bascula;
Adafruit_MPU6050 imu;
Servo servoRodilla;

bool sistemaActivo = true;  // enciende/apaga el interruptor
bool hayCarga = false;
bool referenciaLista = false;
float anguloReposo = 0.0f;
float anguloFiltrado = 0.0f;
float anguloServoActual = SERVO_CENTRO_GRADOS;
uint32_t ultimaLecturaMs = 0;
uint32_t ultimoEstadoMs = 0;
```

Configuración global del sistema: Se incluyen las bibliotecas necesarias, la asignación de pines para los componentes electrónicos y la definición de constantes y variables, como los intervalos de lectura y los umbrales de ángulo, para organizar y facilitar el funcionamiento del programa.

```
float calcularInclinacionAccel(float ax, float ay, float az) {
  // Ángulo del plano X-Z
  return atan2f(ax, sqrtf(ay * ay + az * az)) * 180.0f / PI;
}

bool leerAngulo(float &angulo) {
  sensors_event_t aceleracion, velocidadAngular, temperatura;
  if (!imu.getEvent(&aceleracion, &velocidadAngular, &temperatura)) return false;

  const uint32_t ahora = millis();
  float dt = (ahora - ultimaLecturaMs) / 1000.0f;
  if (dt <= 0.0f || dt > 0.2f) dt = INTERVALO_LECTURA_MS / 1000.0f;
  ultimaLecturaMs = ahora;

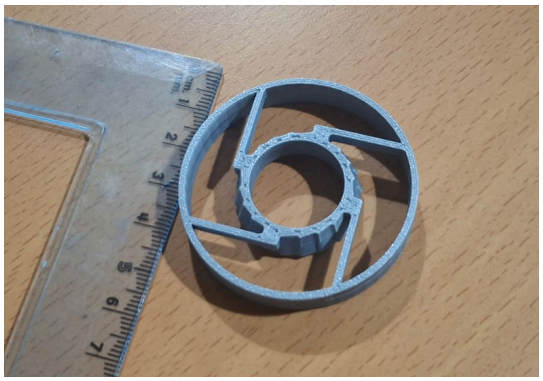
  const float anguloAccel = calcularInclinacionAccel(
      aceleracion.acceleration.x,
      aceleracion.acceleration.y,
      aceleracion.acceleration.z);
  const float gyroXGradosS = velocidadAngular.gyro.y * 180.0f / PI;

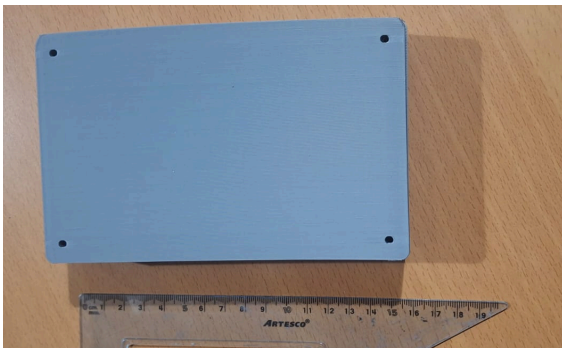
  if (!referenciaLista) {
    anguloFiltrado = anguloAccel;
    anguloReposo = anguloAccel;
    referenciaLista = true;
  } else {
    anguloFiltrado = CONSTANTE_FILTRO_COMPLEMENTARIO *
                     (anguloFiltrado + gyroXGradosS * dt) +
                     (1.0f - CONSTANTE_FILTRO_COMPLEMENTARIO) * anguloAccel;
  }
  angulo = anguloFiltrado - anguloReposo;
  return true;
}
```

Funciones de lectura y cálculo del ángulo: Se implementaron funciones para calcular la inclinación mediante el sensor MPU6050, utilizando datos de aceleración y velocidad angular. Además, se aplica un filtro complementario para mejorar la estabilidad de las lecturas y se resta el ángulo de reposo inicial para determinar el movimiento neto de la articulación.

II. Verificación preliminar del diseño por subsistemas

MECÁNICA Y FABRICACIÓN

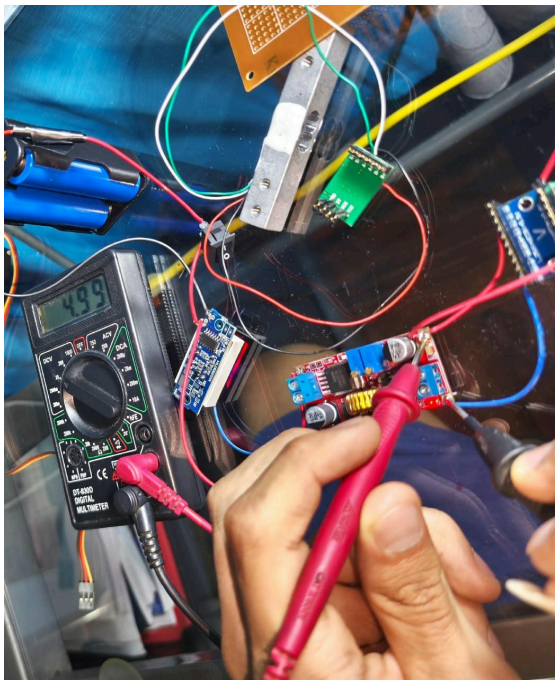




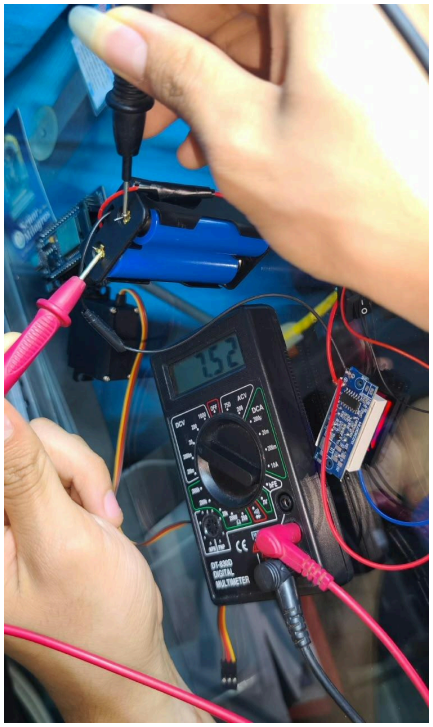


Se realizaron mediciones de las piezas impresas en 3D para verificar sus dimensiones y compararlas con los modelos CAD. Asimismo, se realizó el ensamblaje de la tapa con la caja de control para comprobar su ajuste y la alineación de los agujeros de fijación.

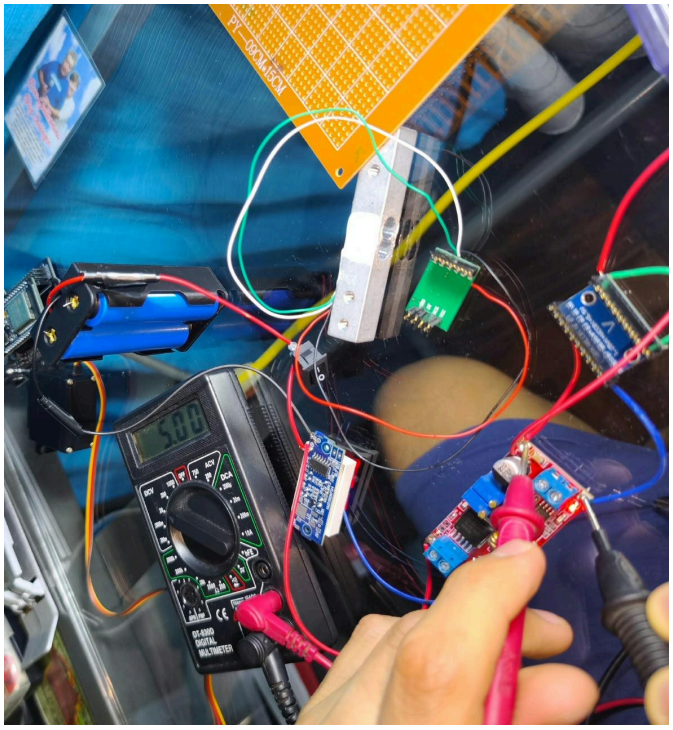
ELECTRÓNICA Y SENSORES



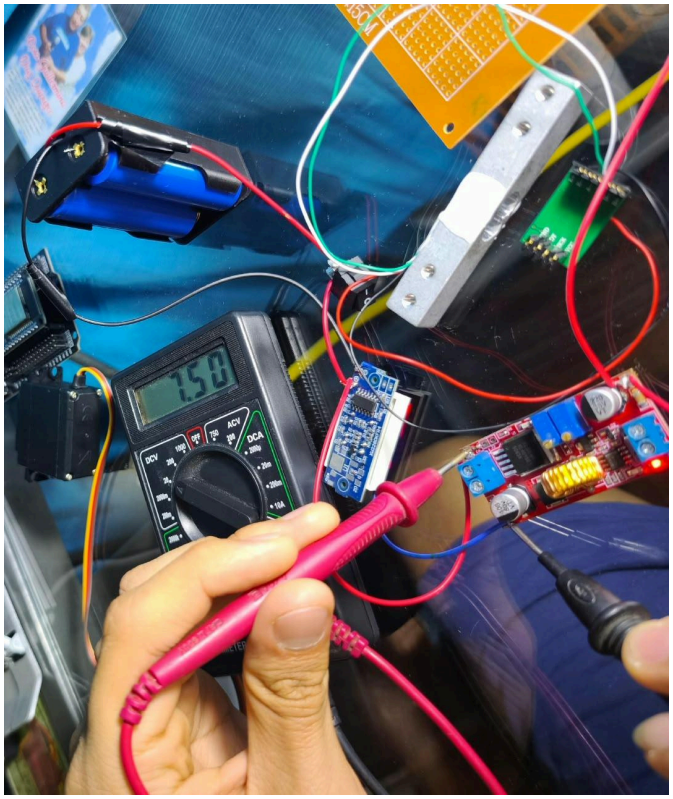
Medición regulador de voltaje (7.52V A 5.0 V)



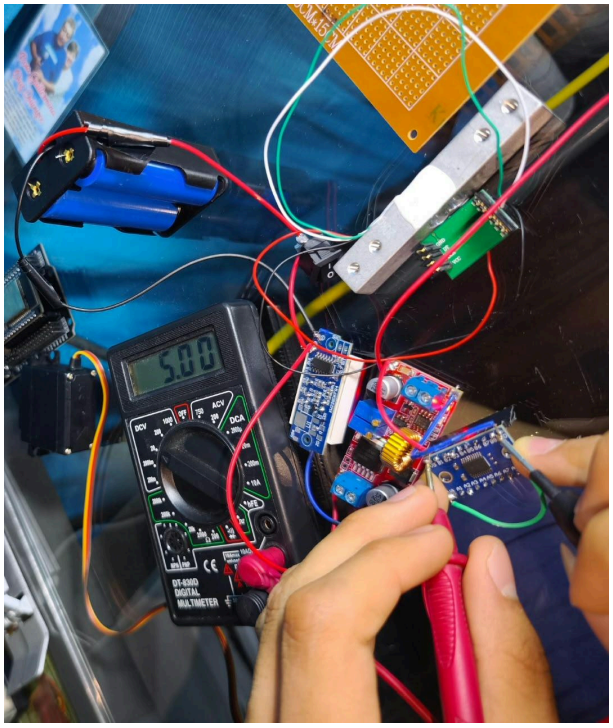
Medición pilas 7.52V



Medición regulador de voltaje (7.52V A 5.0 V)



Medición entrada 7.5V



Medición regulador 5V a 3.3V